

Rachel-v2 系统内部完整说明

本文档是 Rachel-v2 的内部完整系统说明。它面向系统设计复盘、论文方法讨论、代码维护和后续功能扩展，集中说明 Rachel-v2 的目标设定、状态机、action-space、渐进式上下文披露、动态 prompt 注入、validation gate、route audit/export 和路线完成后的 terminal closure audit。

`system_brief_packet` 是简版分享包；本文档保留更多内部机制、代码符号、状态边界和设计理由。分篇文档仍作为可维护版本保留，本文档作为统一阅读稿。

1. 核心定位

Rachel-v2 的目标不是让 LLM 一次性写出一条看似完整的从头合成路线，而是把长程合成规划组织为一系列可执行、可验证、可继承的 route-state transitions。每一步路线选择都必须落到一个当前分子节点、一个真实 reaction site 或 LLM 自提 action、一次 sandbox 验证、一次 commit/accept 审计记录。

这使 Rachel-v2 更接近一种形式化的化学推理工作流。LLM 负责化学判断、机制推理、路线策略和候选比较；代码系统负责分子事实、action-space 构造、状态树维护、验证门控和最终审计。

在路线执行中，Rachel-v2 先分析目标分子的骨架、官能团、电子效应和可断位点，再围绕当前分子生成可验证的候选反应动作。候选动作不会因为被提出就进入路线；它必须经过前体回推、一致性检查和 validation gate，才会被写入 route tree，并作为下一轮推理的分子状态继续推进。

Rachel-v2 的核心表述是：

Rachel-v2 turns retrosynthesis into formal route-state reasoning within an action-space-derived chemical domain, where each LLM decision must be executed, validated, and inherited as the next molecular state.

2. Rachel-v2 要解决的问题

传统 LLM 逆合成回答常见问题包括：

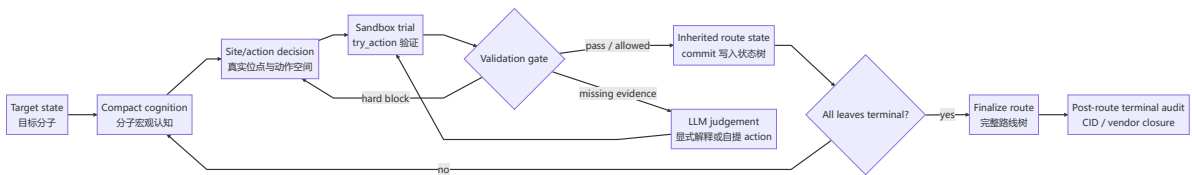
- 路线文本可以很流畅，但每一步是否真的对应当前分子状态不清楚。
- 中间决策缺少可复查证据，后来很难知道为什么选了某个前体。
- 局部模板命中和长程路线质量容易混在一起。
- 没有模板命中时，LLM 容易机械接受高级 terminal 或停止路线设计。
- terminal 是否完成到可接受起始原料，以及这些 terminal 是否有公开记录或供应商证据，往往停留在过宽的理想数据库中。

Rachel-v2 将这些问题变成状态化、结构化、可审计的问题：

- 当前分子事实由 `build_decision_context` 生成。
- 当前可执行动作由 `CandidateUnit` action-space 表达。
- LLM 选择必须通过 `try_action` sandbox。
- commit 写入 route tree，不只是写入聊天记录。
- validation gate 将错误、警告和缺证据分开。
- export/report 保留每一步选择、被拒候选和验证证据。
- route 完成后，所有 terminal 统一进行 PubChem CID 和 vendor evidence 终点审计。

3. 生命周期

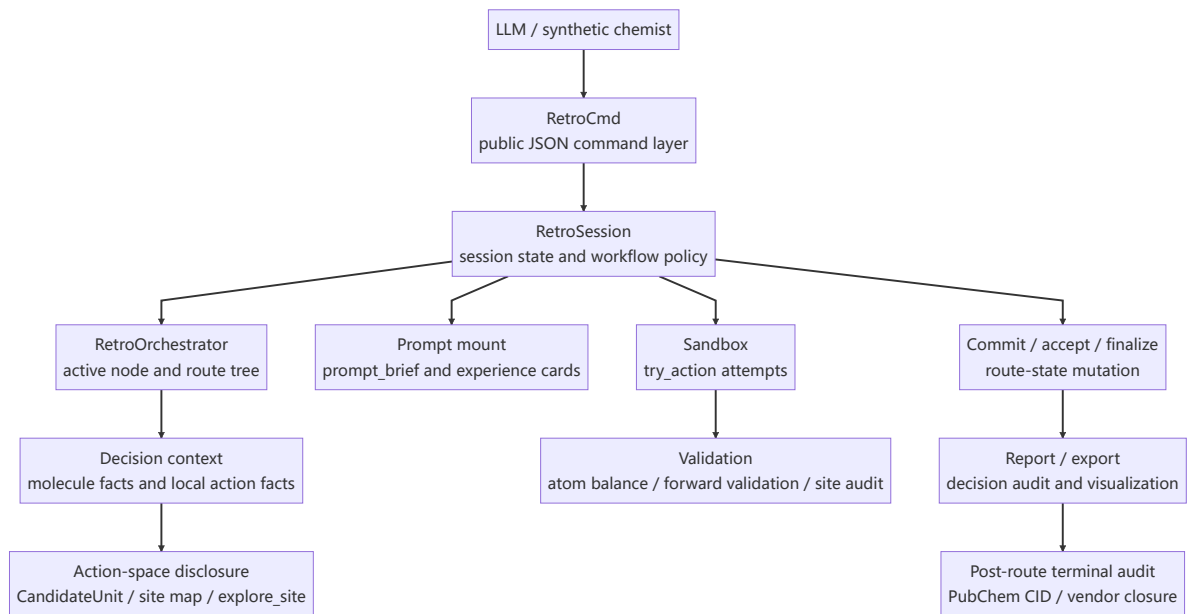
Rachel-v2 的路线生命周期从 target state 开始，经由 compact cognition、site/action decision、sandbox trial、validation gate、commit/accept 和 post-route terminal audit 形成完整闭环。



这个生命周期体现了 Rachel-v2 的核心边界：路线选择发生在当前 active node 上；系统只把当前决策必要的信息暴露给 LLM；完整路线历史由 session 保存；终点可买性和公开记录证据在完整路线形成后统一审计。

4. 架构与模块边界

Rachel-v2 的架构由三层组成：命令接口、路线状态机、化学工具层。LLM 与系统交互时只调用 public commands；系统内部通过 session JSON 保存状态，并由 orchestrator 管理 route tree 与当前 active node。这种结构保证完整路线历史由 session 保存，LLM 每次只处理当前化学节点和当前决策所需证据。



主要代码职责如下：

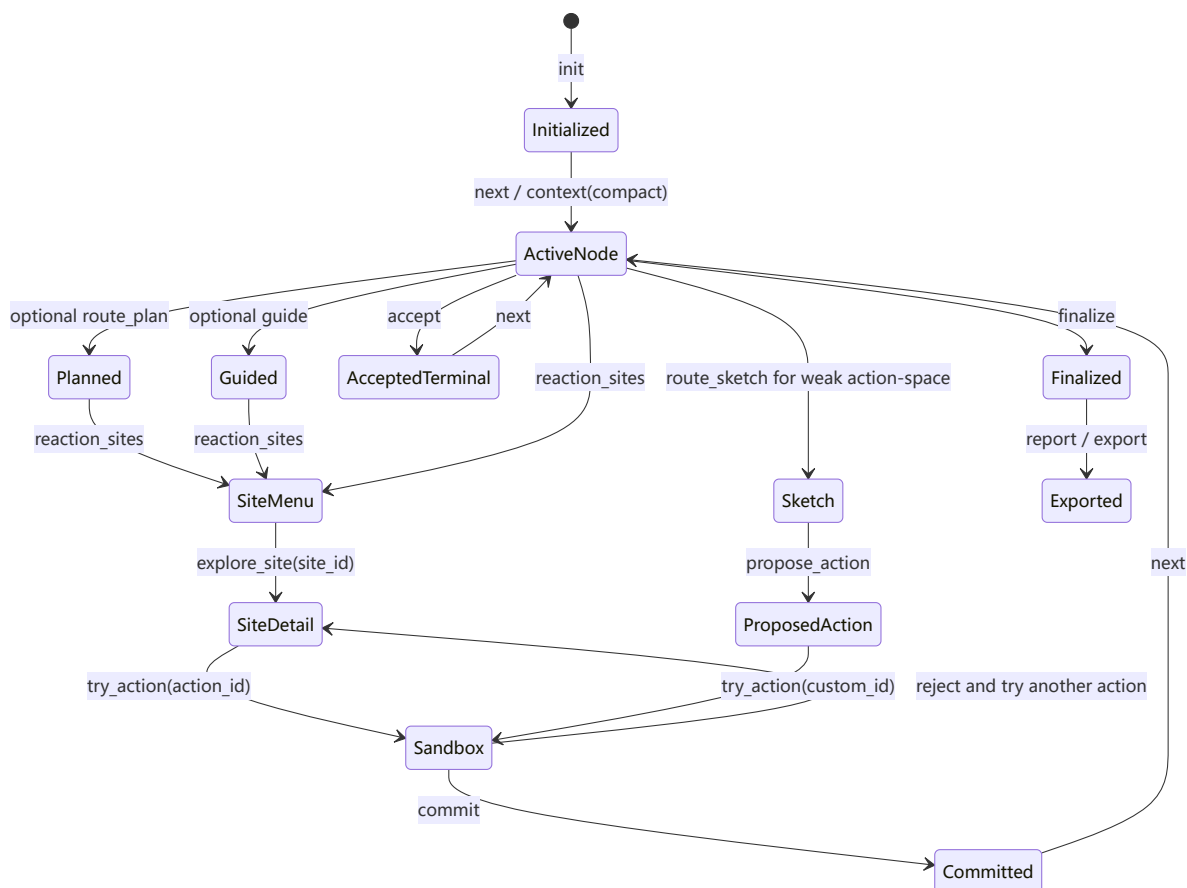
- `Rachel/main/retro_cmd.py::RetroCmd.execute` 是公共命令层。它把命令名分发到 session 方法，并通过 `PUBLIC_COMMANDS` 控制默认暴露给 LLM 的命令集合。隐藏旧命令仍可兼容旧脚本，但不作为主路径。
- `Rachel/main/retro_session.py::RetroSession` 是状态化工作流中心，负责创建和加载 session，并暴露 `guide`、`route_plan`、`route_sketch`、`reaction_sites`、`explore_site`、`try_action`、`commit`、`accept` 等流程动作。
- `Rachel/main/retro_orchestrator.py::RetrosynthesisOrchestrator` 负责当前 active context、route tree 和节点推进，是 `next`、`reaction_sites`、`explore_site`、`commit`、`accept_terminal` 和 `finalize` 的状态基础。

- `Rachel/main/strategy_disclosure.py::CandidateUnit` 统一候选动作表达。
`build_site_reaction_map` 和 `explore_site` 将内部 candidate 转成 LLM 可读的 site-first action-space。
- `Rachel/main/prompt_mount.py::build_prompt_brief` 将内部 prompt mount 投影成短的 LLM 可读提示，避免每一步读取完整经验库或完整策略对象。
- `Rachel/chem_tools/forward_validate.py::build_validation_gate` 将验证结果组织为 hard block、soft warning、override required、missing evidence 和 pass。
- `Rachel/main/retro_report.py::generate_forward_report` 与 `Rachel/main/retro_output.py::export_results` 将路线树、决策审计和终点信息输出为可复查结果。
- `Rachel/tools/pubchem_terminal_audit.py::audit_record` 与 `build_pubchem_metrics` 对路线完成后的 terminal 集合进行 CID/vendor closure audit。

5. 状态机

Rachel-v2 的主流程是状态机，不是一次性文本生成。推荐流程为：

```
init -> next/context(compact) -> route_plan/guide optional -> reaction_sites ->
explore_site -> try_action -> sandbox_list -> commit/accept -> finalize ->
report/export
```



Session JSON 是唯一状态载体。Rachel-v2 不要求 LLM 每一轮完整读取历史 route tree。session JSON 保存完整状态；LLM 默认只读取当前命令返回的短上下文，例如当前 node payload、`prompt_brief`、`reaction_sites`、`explore_site` 或 `sandbox_list` 摘要。

这一区分很关键：

- 完整 session 用于持久化、审计和导出。

- LLM 默认上下文只需要当前 active node 和当前决策所需字段。
- `prompt_brief` 会携带短的 route plan、chemist guidance、route sketch 或经验卡摘要。
- 完整 `session.json`、完整导出报告、`context(full)` 和 diagnostic 不应作为默认 LLM 输入。

每一步化学决策都绑定到当前 active node。如果在旧节点继续试 action，或者把 `reaction_sites` 与 `next` 并行执行，就会得到错误或过期上下文。系统设计把这个约束写入 `RetroSession` 与 `RetroCmd` 的流程中：先 `next` 生成当前 active context，再 `reaction_sites`，再 `explore_site`，再 `try_action`。

6. Action-Space 的定义

Rachel-v2 的 action-space 是当前分子可被系统和 LLM 操作的化学动作域。它不等同于一个固定模板库，也不等同于手写策略组。模板、断键候选、FGI、smart capping 和 LLM 自提前体都可以被规范化为 action，但最终必须通过同一套 sandbox 与 validation 逻辑。

单独说 template 容易造成两个误解：

- 好像系统只是在匹配一张模板表。
- 好像没有模板命中就没有化学路线空间。

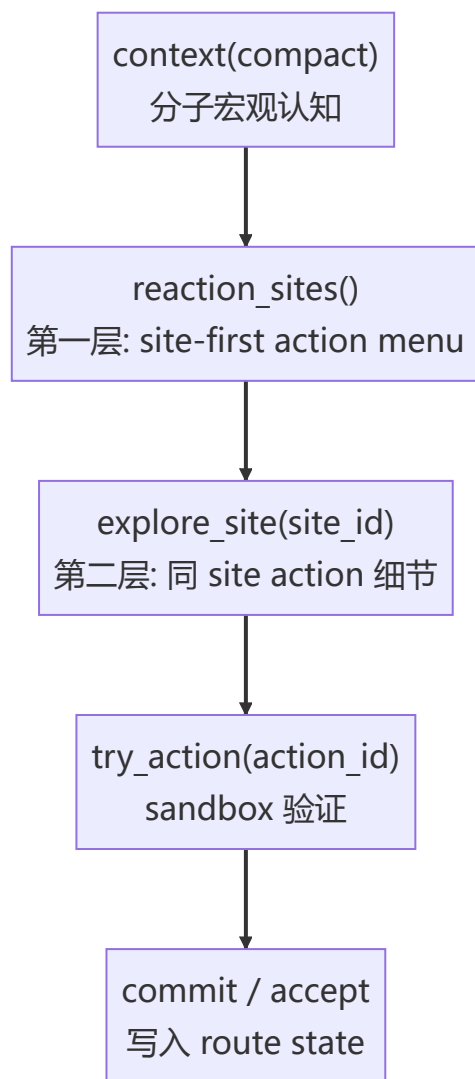
当前代码中的事实更宽。`CandidateUnit` 同时容纳：

- bond disconnection 产生的候选；
- FGI 候选；
- smart capping 候选；
- terminal 候选；
- LLM 通过 `propose_action` 登记的 custom precursor action；
- route sketch 派生的 custom action 元数据。

因此，Rachel-v2 采用 action-space 叙事：模板是 action 的一种来源，action-space 是当前状态下可执行、可验证、可审计的化学动作集合。

7. 渐进式上下文披露

Rachel-v2 使用 compact、第一层、第二层和 sandbox 的渐进式披露，避免让 LLM 在每一步读取完整候选树或完整 session。



7.1 Compact

Compact 面向 LLM 初始认知。它不应内嵌完整第一层菜单，也不应暴露旧 strategy group 或 diagnostic 字段。它的价值是帮助 LLM 建立宏观分子判断：

- 当前分子身份和 brief。
- complexity brief。
- functional group brief。
- warnings。
- reaction opportunity brief，例如 site 数量、action 数量、高竞争位点摘要。
- 下一步命令提示：`reaction_sites()`。

Compact 的目标不是让 LLM 直接决定具体前体，而是让 LLM 知道“这个分子有哪些关键 handle 和值得展开的位点”。

7.2 第一层: reaction_sites

`reaction_sites()` 是 site-first grouped action menu，由 `build_site_reaction_map` 生成。它按真实 reaction site 组织候选，而不是按旧 strategy group 或平铺 reaction list 组织。

每个 site 重点表达：

- `site_id`

- `site_type`
- `site_hint`
- `action_count`
- `reaction_count`
- `competition_hint`
- `risk_hint`
- `reactions`

这一层的价值是让 LLM 快速定位“同一位点有哪些竞争反应”，例如同一个 N-aryl site 下比较 SNAr、Buchwald、Ullmann、Chan-Lam 等，而不是打开多个反应 payload 后手工对齐 site。

7.3 第二层: `explore_site`

`explore_site(site_id)` 返回同一真实 site 下所有 action 细节。代码事实见

`Rachel/main/strategy_disclosure.py::explore_site` 与

`Rachel/main/retro_session.py::explore_site`。

第二层重点保留：

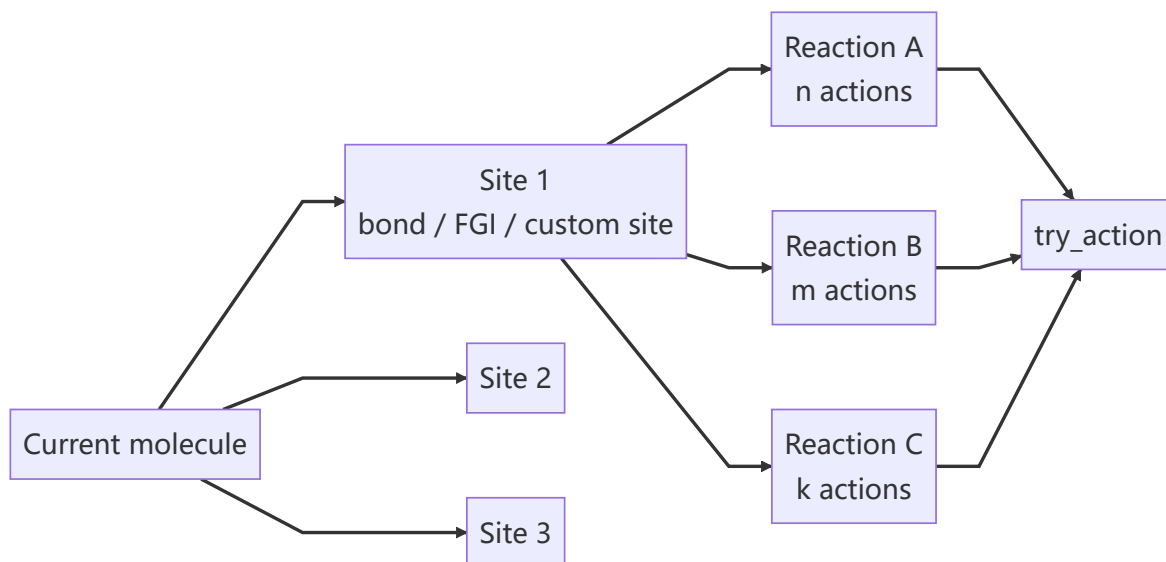
- `action_id`
- `site_id`
- reaction id/name
- source
- precursor preview
- `risk_tags`
- `bond_idx` / `actual_bond_idx` / `fgi_idx`
- `atoms`
- `role_pair`
- `bond_type`
- `in_ring`
- `bond_fg_context`

这些字段支持 LLM 判断反应位点、前体是否合理、是否保留核心骨架、是否存在同 site 竞争动作。

7.4 Sandbox: `try_action`

`try_action(action_id)` 是主 sandbox 入口。它将 action-space 的候选送入统一执行和验证过程。

旧的 `try_bond`、`try_fgi`、`try_precursors` 等入口可为兼容保留，但不应作为默认 LLM 路径。



旧的 `explore_bond` 或 `strategy group` 思路更像“先由脚本人为组织候选”。Rachel-v2 的方向是让第一认知层由真实 reaction site 组织，避免手写 `strategy group` 抢占 LLM 的化学判断入口。

这并不意味着系统放弃规则和经验，而是把规则放到正确层级：

- site/action 由分子事实和候选生成产生；
- prompt brief 和经验卡提供短提醒；
- LLM 负责化学质量判断；
- validation/audit 记录证据；
- diagnostic 不进入默认 LLM 上下文。

8. 动态 Prompt 注入与策略控制

Rachel-v2 的 prompt 机制不是把 `SKILL.md`、`workflow.md`、`experience.md` 或完整 session 每轮塞给 LLM。默认 LLM 可读对象是 `prompt_brief`：它是由完整 prompt mount 投影出来的短上下文，只保留当前阶段可执行的提醒和经验卡文本。

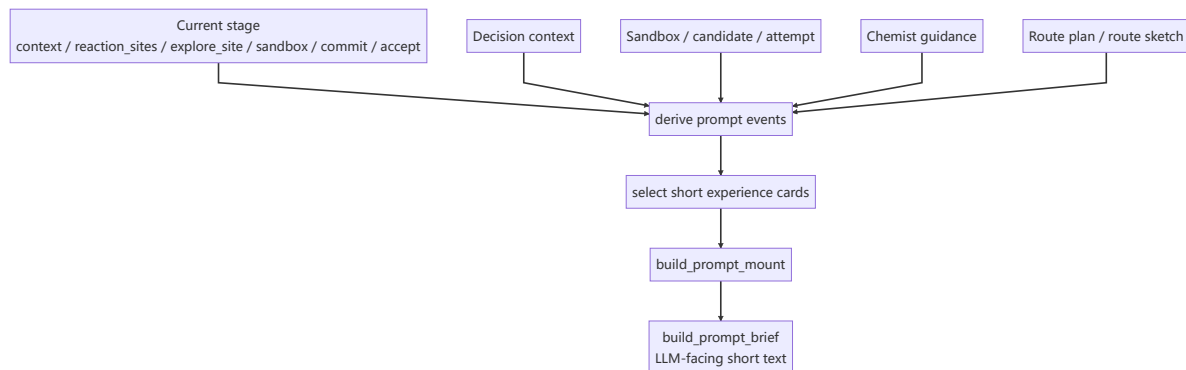
这些动态提示不会替代化学判断，也不会直接改变路线树。复杂目标、弱 action-space、advanced terminal 或合成专家指导只会改变当前阶段的注意力分配；真正改变路线的动作仍然是 `try_action` 后的 `commit` 或 terminal `accept`。

`Rachel/main/prompt_mount.py::build_prompt_mount` 构建内部 mount。它包含 stage、events、standing rules、quality guardrails、command policy、experience cards、chemist guidance、route plan brief、route strategy brief 等结构。

`Rachel/main/prompt_mount.py::build_prompt_brief` 将 mount 投影为 LLM 默认可读 payload。它会保留：

- `stage`
- `events`
- `next_actions`
- `quality_guardrails`
- `active_experience_card_ids`
- `experience_prompts`

- `chemist_guidance`
- `route_plan_brief`
- `route_strategy_brief`
- `self_prompt`



这个设计的目的很明确：让经验和规则参与动态流程，但避免每一步重复注入完整文档和完整 JSON object。

9. 长期策略、局部救援与合成专家指导

`RetroSession.route_plan` 保存短的全局路线 thesis。它不是 `commit`，也不会直接改变 `route tree`。它用于让 LLM 在长程逆合成中维持一个可修订的主策略，例如 `late FGI`、`scaffold assembly`、`electronic-state strategy` 或 `hybrid strategy`。

`route_plan` 的价值是让全局策略长驻，但保持短。每当 `action-space`、`terminal review`、`sandbox evidence` 或 `chemist guidance` 表明原策略隐藏了真实核心问题时，LLM 应该修订 `route plan`，而不是只在 `commit reasoning` 中临时改口。

`RetroSession.route_sketch` 是局部救援工具。它用于系统 `action-space weak`、候选 `route-incoherent` 或 `advanced terminal` 可能还能被机理驱动拆解时。

关键边界：

- `route sketch` 是 `strategy only`，不改 `tree`。
- 它不能绕过 `sandbox`。
- 如果 `sketch` 找到可信下一步，必须落成 `propose_action -> try_action -> commit`。
- 如果完全无法定义可信一步 `action`，才能以明确理由接受 `advanced terminal`。

这使 LLM 能发挥自己的合成能力，但仍受状态机和验证约束。

`RetroSession.guide` 允许合成专家用自然语言指向当前 `active node` 的处理方向。它会持久化原始指导文本用于 `audit`，但默认 `prompt` 只读取短摘要。

这不是第三种独立模式，也不是绕过 LLM。它是一种动态上下文注入：

- 合成专家可以指出反应位点、优先反应、前体方向或约束。
- LLM 仍需用 `reaction_sites -> explore_site -> try_action` 或 `propose_action -> try_action` 验证。
- `route tree` 只有 `commit` 或 `accept` 才会改变。

三类策略信号的区别如下：

信号	作用	是否改 tree	默认给 LLM 的形式
<code>guide</code>	合成专家自然语言方向	否	<code>chemist_guidance</code> 短摘要
<code>route_plan</code>	长期全局路线 thesis	否	<code>route_plan_brief</code>
<code>route_sketch</code>	弱 action-space 或 advanced terminal 的局部救援	否	<code>route_strategy_brief</code> / event card

Prompt 设计原则：

- 常驻规则短而硬：化学质量、骨架守恒、原子来源、验证优先。
- 经验卡按 stage/tag/event 触发，不全量加载。
- 对特殊反应的卡可以存在，但必须短。
- 复杂策略进入 `route_plan` 或 `route_sketch`，不要变成每轮全文粘贴。
- `prompt_brief` 是 LLM 默认读取对象，完整 mount 是内部审计对象。

10. Validation Gate

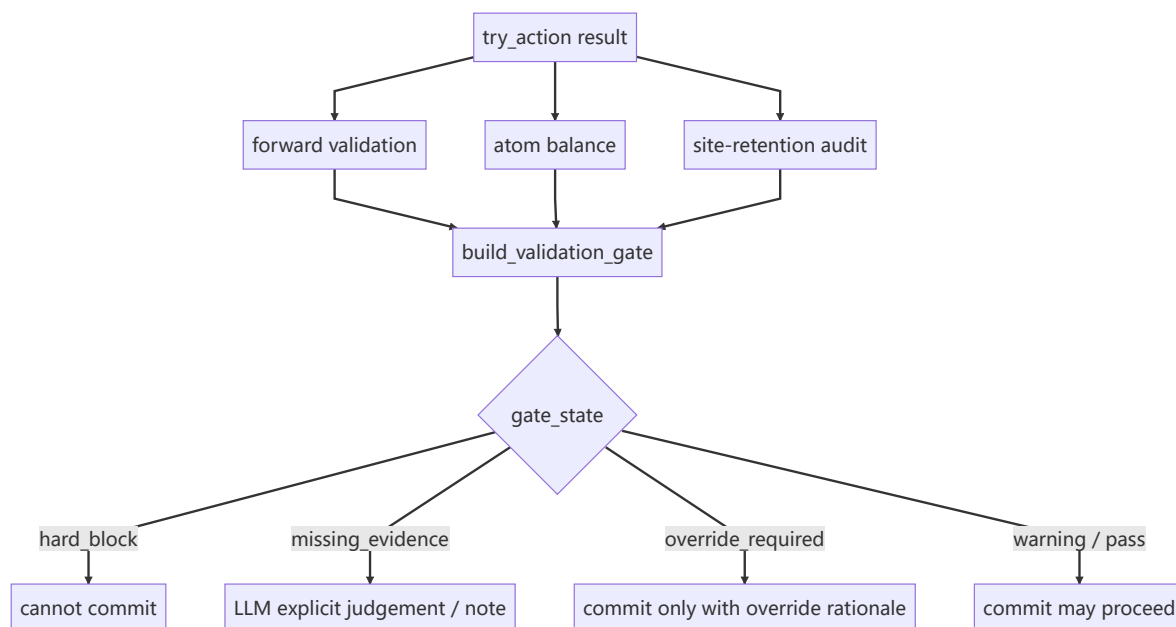
Rachel-v2 的路线质量不只靠 LLM 自信判断，也不只靠模板分数。系统把 sandbox、forward validation、site-retention、atom accounting、commit audit 和最终 export/report 组合成可审计闭环。

Validation gate 在提交路线前拆分不同证据来源：原子和骨架是否守恒、反应位点是否一致、forward template 失败是否来自化学错误或模板覆盖不足，以及 LLM 是否给出足够清楚的机理解释。

`Rachel/chem_tools/forward_validate.py::build_validation_gate` 将验证结果分成几类：

- `hard_block`
- `soft_warning`
- `override_required`
- `missing_evidence`
- `pass`

这一分类比简单的 pass/score 更能支持化学路线规划，因为真实情况经常是“系统证据不足”，而不是“化学上错误”。



模板库覆盖不完整时，forward template 可能不能重新生成 target。Rachel-v2 不把这类情况直接当作 hard block。代码注释中也明确：template execution misses and missing template IDs are inconclusive evidence, not chemical hard blocks.

这种保守门控避免把“模板缺证据”误判为“化学错误”。但它也要求 LLM 在 commit 时写清楚：

- 机理为什么仍可信；
- 位点是否保留；
- 骨架是否守恒；
- 原子来源是否合理；
- 被拒候选为什么更差。

11. Decision Audit 与 Export

`RetroSession.commit` 在提交 sandbox attempt 时，会把候选比较、selected action、rejected alternatives、validation gate、prompt cards、decision source 等信息写入决策审计。

`Rachel/main/retro_report.py::generate_forward_report` 和相关格式化函数会把这些审计信息输出到最终报告。

Decision audit 是 Rachel-v2 的核心证据链：



`Rachel/main/retro_output.py::export_results` 生成：

- visual report;
- `report.txt`;
- `tree.json`;
- `tree.txt`;
- `terminals.json`;
- `visualization.json`;
- `session.json`。

这些文件的角色不同：

- `report.txt` 和可视化报告用于人工阅读。
- `tree.json` 和 `session.json` 用于审计和复现。
- `terminals.json` 是路线完成后 terminal closure audit 的输入。
- `visualization.json` 支持 route graph 可视化。

决策审计的价值在于保存路线形成过程，而不只是保存最终结果。它记录系统如何从候选空间进入 LLM 判断，再进入验证和路线继承，避免最终输出变成一条缺少形成依据的合成路线。

12. 路线完成后的 Terminal Closure Audit

终点可买性不是每一步 action 选择的在线筛选标准。所有 terminal 合成完成以后，再统一执行 PubChem CID/vendor closure audit，从而观察整条路线是否真正闭合到公开可追踪的起始原料集合。

Terminal closure 是路线完成后的终点审计层，不是路线执行过程中的在线可买性过滤器。Rachel-v2 先通过 action-space、sandbox、validation gate、commit audit 和 terminal/advanced-terminal rationale 完成路线树；随后对 `terminals.json` 中的所有 terminal 统一执行 PubChem CID/vendor closure audit。

`Rachel/tools/pubchem_terminal_audit.py::audit_record` 对 terminal record 进行 PubChem 查询和本地 allowlist 处理。`build_pubchem_metrics` 输出两个正式指标：

- `pubchem_cid_closed`：存在 PubChem CID 或 allowlist 提供等价 CID closure evidence。
- `vendor_closed`：存在 PubChem Chemical Vendors evidence 或 allowlist 提供 vendor closure evidence。

Allowlist 不是第三个指标。它是证据来源，用来处理简单离子、常见试剂、现制备物种或代表形式查询困难的问题。最终仍归入 `pubchem_cid_closed` 与 `vendor_closed` 两个指标。

Terminal closure 不等于真实采购订单，也不等于实验可行性验证。它也不是 LLM 在每一步决定是否继续拆解的唯一标准。它的含义更精确：

- CID closure 表示公开化学记录可追踪。
- vendor closure 表示 PubChem vendor evidence 支持商业供应信号。
- allowlist closure 表示简单试剂或离子可由更基础、可买的代表试剂或来源物解释。

这使 Rachel-v2 的终点判断从“LLM 觉得可以买”转成路线完成后的可复查公开证据。该设计更能揭示系统的真实合成能力：先看 Rachel-v2 能否形成化学合理、状态连续、terminal 合成闭合的完整路线，再用 CID/vendor audit 评价这组 terminal 起始原料的公开闭合程度。

13. Case 证据链

Rachel-v2 的真实运行结果由一条结构化证据链表达：目标分子进入状态机，系统生成当前分子的 action-space，LLM 做化学判断，sandbox 和 validation gate 提供执行证据，commit 将选择写入 route tree，路线完成后再统一执行 terminal closure audit。

每个 case 同时保留“系统给出了什么候选”和“LLM 为什么选这个候选”。因此，最终路线不是黑箱答案，而是一串可复查的化学决策。

已有轻量 case 记录：

- `docs/action_space_case_record_n1_1391.md`

一个 Rachel-v2 case 由六类证据组成：

1. Target and challenge

目标分子、关键官能团、核心合成难点。

2. Initial cognition

compact 层给出的宏观分子认知，包括复杂度、关键 handle、高竞争位点和 warnings。

3. Action-space exposure

第一层 `reaction_sites()` 按真实位点组织候选；第二层 `explore_site()` 展开同 site 竞争 action。

4. LLM chemical decision

LLM 对 action 的化学选择理由和被拒候选理由。核心证据是机理合理性、骨架守恒、前体可推进性、路线收敛性和 terminal 目标。

5. Sandbox and validation

`try_action` 返回前体、sandbox evidence、validation gate 状态，以及 missing evidence 或 warning 的解释。

6. Route-state inheritance and post-route closure audit

`commit` 生成新的 precursor node 并继承为下一轮 route state。所有 terminal 节点形成以后，系统再对 `terminals.json` 统一执行 PubChem CID/vendor terminal closure audit。



这一机制说明：LLM 的化学推理并未被模板替代，而是被放入一个可执行、可验证、可继承、可审计的路线状态机中。

主体叙述应纳入的信息包括：

- 目标分子图或路线树图。
- compact 中的 molecule/complexity/functional group brief。
- 高竞争 site 的 `reaction_sites` 摘要。
- `explore_site` 的同 site action 对比。
- `try_action` 的 validation gate 摘要。
- commit reasoning 和 rejected alternatives 摘要。
- 路线完成后的 `pubchem_cid_closed` 与 `vendor_closed` 终点审计结果。

主体叙述不应纳入的信息包括：

- 完整 `session.json`。
- 完整 `workflow.md`。
- 完整 `prompt_mount` object。
- 大量历史 sandbox attempts。
- diagnostic payload。

高质量 case 应同时体现两类证据：

- Rachel-v2 action-space 提供了合理的化学动作基底，例如多个同 site 竞争 action 或 route-sketch/custom action 派生动作。
- LLM 在该动作空间中发挥了化学推理能力，例如拒绝分数较高但路线质量较差的候选、处理 missing evidence、或提出经过 sandbox 验证的自提 action。

路线完成后的 terminal closure audit 提供终点公开证据。它不替代路线执行过程中的化学判断，也不作为每一步 action 选择的在线可买性标准。

14. 代码事实边界

本文档依据当前 Rachel-v2 代码事实整理，主要证据路径包括：

- `Rachel/main/retro_cmd.py::RetroCmd.execute`
- `Rachel/main/retro_session.py::RetroSession`
- `Rachel/main/retro_orchestrator.py::RetrosynthesisOrchestrator`
- `Rachel/main/strategy_disclosure.py::CandidateUnit`
- `Rachel/main/strategy_disclosure.py::build_site_reaction_map`
- `Rachel/main/strategy_disclosure.py::explore_site`
- `Rachel/main/prompt_mount.py::build_prompt_mount`
- `Rachel/main/prompt_mount.py::build_prompt_brief`
- `Rachel/chem_tools/forward_validate.py::build_validation_gate`
- `Rachel/main/retro_report.py::generate_forward_report`
- `Rachel/main/retro_output.py::export_results`
- `Rachel/tools/pubchem_terminal_audit.py::audit_record`
- `Rachel/tools/pubchem_terminal_audit.py::build_pubchem_metrics`

15. 与简版分享包的关系

`system_brief_packet` 是面向交流的简版包，隐藏内部代码符号、命令协议、schema 细节和运行路径。它强调系统思想、路线状态推理、action-space、validation/audit 和 terminal closure 的概念价值。

本文档是内部完整说明。它保留更多机制细节，用于：

- 系统设计复盘；
- 论文方法部分展开；
- 代码维护和功能扩展；
- case 证据链构建；
- prompt、action-space、validation gate 和 report/export 的后续优化。

两者的关系不是新旧版本关系，而是信息边界不同：简版用于展示系统思想，内部完整说明用于维护系统事实和设计理由。